

# Benchmarking MCP Vulnerability Detection from a Pairwise Correctness Perspective

Anonymous Author(s)

## Abstract

Model Context Protocol (MCP) servers have become essential for connecting large language models (LLMs) with external tools, data sources, and execution environments. However, their privileged access and bridging role introduce significant security risks. Despite many LLM-based detection methods, it remains unclear whether they can reason about vulnerabilities, distinguishing vulnerable from fixed MCP servers rather than just identifying risky API calls. In this work, we present a dedicated benchmark for pairing MCP server vulnerability detection, comprising 75 paired vulnerable and fixed MCP tools. We adopt the Matched Pair Accuracy (MPA) metric to evaluate detection methods, exposing performance gaps relative to precision and recall, particularly on complex or novel vulnerabilities. Our benchmark provides a new perspective for systematic, reproducible, and MCP-specific security evaluation.

## CCS Concepts

• Software and its engineering → Software libraries and repositories; • Security and privacy → Software security engineering.

## Keywords

MCP, vulnerability, detection, benchmark

## ACM Reference Format:

Anonymous Author(s). 2018. Benchmarking MCP Vulnerability Detection from a Pairwise Correctness Perspective. In *Companion Proceedings of the 34th ACM Symposium on the Foundations of Software Engineering (FSE '26)*, June 5–9, 2026, Montreal, Canada. ACM, New York, NY, USA, 5 pages. <https://doi.org/XXXXXXX.XXXXXXX>

## 1 Introduction

As LLM-powered systems are increasingly deployed in real-world environments, developers require reliable mechanisms to connect models with external tools, data sources, and execution environments. MCP servers have emerged as an ideal solution. Their open design and compatibility with diverse development ecosystems have accelerated adoption across open-source communities and industrial platforms.

However, the increasing adoption of MCP servers has also expanded their attack surface, exposing them to a variety of security threats. As middleware components that bridge LLMs with

external tools, file systems, and environments, MCP servers often operate with elevated privileges and direct access to sensitive resources, making them attractive targets for adversaries. For example, the Markdownify MCP Server exposed the improper input handling [16] that allowed crafted payloads to trigger unintended command execution. The case illustrates that, despite their functional benefits, MCP servers remain insufficiently hardened and are increasingly becoming a critical security bottleneck in the LLM application supply chain.

Recent research and industry practices have begun to develop targeted vulnerability detection methods for MCP servers [4, 9, 10, 20, 25]. Nevertheless, despite the emergence of various detection techniques, the effectiveness of MCP server vulnerability detectors remains largely unclear due to the absence of standardized evaluation criteria and representative datasets.

To address this gap, we propose a dedicated benchmark for MCP server vulnerability detection. Our benchmark constructs from real-world MCP server vulnerability types, including remote code execution (RCE), denial of service (DoS), SQL injection (SQLi), insecure file operations (IFO) and sensitive information disclosure (SID). It comprises 75 vulnerable MCP servers and 75 corresponding vulnerability-fixed MCP servers as paired counterparts.

We conducted an evaluation of existing MCP server vulnerability detection methods using the curated dataset. We adopt the Matched Pair Accuracy (MPA) metric to evaluate detection methods, which highlights the limitations of existing approaches more effectively than traditional precision and recall. This evaluation highlights both the strengths and limitations of current detection techniques, demonstrating the effectiveness of our benchmark in uncovering their shortcomings and identifying areas for improvement. We have released the benchmark and the replicating artifact in [1]. Overall, our paper makes the following contribution:

- We propose PAIREDMCPVUL, an LLM-generated benchmark guided by manually crafted rules covering five common vulnerability types for MCP vulnerability detection.
- We propose the Matched Pair Accuracy for paired vulnerable and fixed MCP tools, providing a more precise measure of a detector’s understanding of vulnerable code than standard metrics.

## 2 Benchmark

### 2.1 Benchmark Construction

To construct our benchmark, we apply a five-step process. The containing samples are of the Python language, which is the dominant language in the current MCP ecosystem (95% of the ecosystem share) [8].

**(1) Vulnerability Type Collection.** To understand the prevalent vulnerability types in existing MCP-based systems, we analyzed publicly disclosed vulnerabilities in the National Vulnerability Database (NVD) [15]. Using the keyword “MCP”, we identified 34

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](mailto:permissions@acm.org).

Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06

<https://doi.org/XXXXXXX.XXXXXXX>

vulnerable MCP servers. Our analysis reveals four dominant vulnerability categories: remote code execution (RCE), denial of service (DoS), SQL injection (SQLi), and insecure file operations (IFO). In addition, we reference the OWASP OWASP LLM Top 10 [19] to complement our classification. Given that MCP servers typically have access to sensitive data and external resources, we additionally include sensitive information disclosure (SID) as a relevant risk category. The remaining nine risk types in the OWASP LLM Top 10 are excluded, as they are not directly applicable to MCP server architectures.

(2) **Vulnerability-Triggering Expression Collection.** We collect vulnerability-triggering expressions (e.g., function calls) corresponding to the five identified vulnerability types.

- **Remote Code Execution (RCE).** We construct RCE vulnerabilities by introducing unsafe system shell invocations and dynamic code evaluation calls into the MCP codebase. For system shell invocation, we select calls including `os.system()`, `subprocess.run(shell=True)`, `subprocess.call()`, and `asyncio.create_subprocess_shell()` with unsanitized user input, thereby enabling direct shell command injection. In addition, we incorporate the use of ctypes to invoke the underlying `libc.system()` function, further expanding the attack surface at the native C++ library layer. For dynamic code execution, we introduce calls to `exec()`, `eval()`, `timeit.timeit()`, and `runpy.run_path()` to construct scenarios that permit arbitrary Python code execution. Furthermore, we include expression evaluation through `pandas.query()`, which can also lead to code injection if improperly handled.
  - **Sensitive Information Disclosure (SID).** We construct scenarios in which MCP servers may access and expose sensitive information from external data sources. Specifically, we simulate cases involving the retrieval of email contents (i.e., the MCP tool directly feeds unsanitized email content into the LLM's context through a return statement) and log files, as well as the copying of sensitive data from private locations to publicly accessible directories (`shutil.copy()`).
  - **Insecure File Operation (IFO).** Insecure file operations (IFO) include path traversal and improper file permission management. For path traversal, we construct attacks using the unfiltered `open()` function with `../` sequences or URL-encoded patterns to bypass directory constraints. We also exploit path-overwriting in `os.path.join()` for absolute paths, resolution inconsistencies in `pathlib.Path()`, and unintended file enumeration via wildcards in `glob.glob()`. Improper file permissions include unauthorized deletion (`os.remove()`, `shutil.rmtree()`), insecure permission changes (`os.chmod()`), arbitrary reading (`linecache.getline()`), unrestricted symlink traversal (`os.walk()`), and TOCTOU race conditions between link validation (`os.path.islink()`) and subsequent file access (`open()`).
  - **SQL Injection.** We include the standard `execute()` call, the primary query interface for major relational databases (e.g., MySQL, PostgreSQL, SQLite, and Microsoft SQL Server). We further extend our coverage to widely used ORM and data-access frameworks, including SQLAlchemy's raw SQL interface `sqlalchemy.text()`, Pandas' direct query execution via `pandas.read_sql()`, MongoDB's `find()` operation, and Redis Lua script evaluation through `redis.eval()`.
  - **Denial of Service (DoS).** DoS attacks target computational (CPU), memory, and storage resources. For CPU exhaustion, we model vulnerable regular expressions (e.g., `re.match()`, `re.compile()`) prone to ReDoS, and unbounded computations such as `math.factorial()` or deep recursion without input limits, causing potential stack overflows. Memory exhaustion includes reading an expanded ZIP bomb (`zipfile.ZipFile.read()`) and unconstrained string multiplications. Disk exhaustion is simulated via file descriptor leaks (repeated `open()` without `close()`) and uncontrolled `f.write()` operations. At the business logic level, we model infinite loops, including non-terminating pagination (returning a new `uuid.uuid4()` as `next_page_token`), endlessly expanding directory trees, and circular authentication deadlocks.
- (3) **Varying Context Generation.** We apply multiple varying contexts surrounding the vulnerability-triggering expressions to ensure the robustness and generalization ability of the dataset in complex real-world scenarios.
- **Encoding Variants.** We incorporate multiple encoding techniques to evaluate detector robustness. Specifically, we apply hexadecimal (Hex) and Base64 encodings, which are commonly used to represent binary data in text-based formats and to standardize cryptographic hash outputs. These transformations allow us to assess whether detectors can correctly identify semantically equivalent vulnerabilities expressed in encoded variants.
  - **Dynamic Invocation.** We employ Python reflection to dynamically resolve and invoke system functions, such as constructing system call references through `getattr()`, thereby obscuring direct API usage.
  - **Incomplete defenses.** We construct samples with incomplete defenses to mimic common developer biases. For RCE and SQLi, MCP servers use fragile single-pass blacklist sanitization (e.g., `replace()` without recursion), failing against double-write nesting or mixed-case obfuscation. For insecure file operations, URL-decoded inputs undergo single-pass traversal removal (e.g., `unquote() + replace('./', '')`) and imprecise `startswith()` checks on non-normalized paths, ignoring canonicalization issues. Finally, covert fallback vulnerabilities occur when strict checks (e.g., `int()` conversion) fail, and the server reverts to unsafe operations like f-string SQL queries.
- (4) **Vulnerable MCP Server Generation.** We employ Gemini-1.5-Pro [7] as the base model and design a structured prompt template incorporating persona configuration (Python Security Expert), core logic parameterization, and JSONL output constraints. The template guides the model to generate heterogeneous vulnerability samples in batch, ensuring compliance with MCP protocol specifications. Specifically, it dynamically instantiates predefined vulnerability categories, trigger logic, and variant techniques to produce diverse yet controlled risk scenarios. The complete prompt template is illustrated in Fig. 1.
- (5) **Vulnerable MCP Server Fixing.** We fix MCP vulnerabilities by type. For RCE, we remove user-controlled execution flows, replacing shell-based APIs (e.g., `os.system()`) with parameterized subprocess calls (`shell=False`) and substituting dynamic execution primitives (e.g., `exec()`, `eval()`) with predefined dispatch logic.

**System Persona:** You are an expert Python security researcher and MCP (Model Context Protocol) developer. Your task is to generate a high-quality, executable Python code snippet containing a specific security vulnerability.

**Input Context:**

**Vulnerability Type:** {Vulnerability Type}

**Vulnerability-Triggering Expression:** {expr}

**Varying Context:** {VARIANT}

**Constraint Checklist:**

- Implementation:** Use from mcp.server.fastmcp import FastMCP. Define a tool using the @mcp.tool() decorator.
- API:** Include the standard create\_starlette\_app and uvicorn entry point to ensure the code is runnable as an MCP server.
- Functionality:** The code should look like a legitimate functional tool (e.g., a system utility, a data analyzer), but fundamentally contain the specified vulnerability logic included with the varying context.
- Self-Containment:** Ensure all necessary imports are present.

**Output Format:**

You must output ONLY a single valid JSON object on a single line, following this schema: <JSON SCHEMA>

Figure 1: Prompt for Vulnerable MCP Server Generation

Table 1: Statistics of PAIREDMCPVUL

| Vulnerability Type               | Vulnerable MCP Tool Count | Vulnerability-Fixed MCP Tool Count |
|----------------------------------|---------------------------|------------------------------------|
| remote code execution            | 15                        | 15                                 |
| denial of service                | 15                        | 15                                 |
| SQL injection                    | 15                        | 15                                 |
| insecure file operations         | 15                        | 15                                 |
| sensitive information disclosure | 15                        | 15                                 |
| Total                            | 75                        | 75                                 |

Unsafe deserialization (e.g., pickle) is replaced with data-only formats (e.g., JSON), and strict input validation with canonicalization confines external data. For sensitive information disclosure, we enforce separation between data and instructions via structured JSON, role-label sanitization, parameter whitelists, and bounded outputs. Insecure File Operations are mitigated with a “normalize-then-validate” sandboxing: paths are canonicalized (abspath(), resolve()) and checked against a base directory, with symlink validation, multi-pass decoding, and secure temporary file APIs. For DoS, we enforce deterministic limits on recursion, iteration, memory, and decompression, optimize regex patterns against ReDoS, use non-blocking asynchronous execution, and set explicit termination conditions to prevent infinite loops. SQLi is prevented via parameterized queries, whitelisted identifiers, strict type validation, and elimination of unsafe fallback logic, ensuring data is isolated from executable commands.

**Manual Validation.** Two authors independently conducted a cross-validation and identified 10 erroneous entries out of the 150 implementations. Four vulnerable entries were found unexploitable in practice. We revised their logic to ensure exploitability. The remaining six vulnerability-fixed entries contained flawed patches, either due to overly strict sanitization that broke functionality or incomplete mitigations still bypassable. These patches were refined accordingly. All revised entries then underwent a second independent review until both authors unanimously confirmed their correctness.

Table 2: Experiment Results

| Model            | Precision | Recall | F1-Score | MPA   |
|------------------|-----------|--------|----------|-------|
| Claude Haiku-4-5 | 0.932     | 0.920  | 0.926    | 0.853 |
| DeepSeek-V3-2    | 0.866     | 0.947  | 0.905    | 0.800 |
| GPT-5-mini       | 0.775     | 0.920  | 0.842    | 0.653 |
| Bandit           | 0.688     | 0.293  | 0.411    | 0.227 |
| Semgrep          | 0.800     | 0.053  | 0.100    | 0.053 |
| AI-Infra-Guard   | 0.333     | 0.2667 | 0.296    | 0.107 |

## 2.2 Statistics

Across the five vulnerability categories, we collect 42 vulnerability-triggering expressions corresponding to the identified risk types. We further embed these expressions into 3 varying context settings to simulate realistic usage scenarios. In total, we generate 75 vulnerable MCP tools, along with 75 corresponding fixed versions as paired counterparts. The detailed distribution across vulnerability types is summarized in Table 1.

## 2.3 Metrics

**Precision and Recall.** Precision evaluates the proportion of true vulnerabilities among those identified by the model. Recall reflects the proportion of correctly detected vulnerabilities among all ground-truth vulnerabilities. F1-score measures the harmonic mean of precision and recall, providing a balanced assessment of the model’s detection performance.

**Matched Pair Accuracy.** Traditional precision and recall metrics cannot fully capture whether the model genuinely distinguishes between positive and negative samples based on underlying code semantics, or merely relies on superficial matching of specific API calls. Leveraging our paired benchmark, where each vulnerable sample is matched with its fixed counterpart, we employ Matched Pair Accuracy (MPA) [13]. A pair is deemed correct only when both instances are classified correctly. MPA thus captures the proportion of pairs for which the model simultaneously identifies the vulnerable and non-vulnerable samples correctly.

## 3 Experiments

### 3.1 Experiment Setup

**Evaluated Baselines.** We evaluate a diverse set of baselines, ranging from state-of-the-art LLMs to industry-standard security analysis platforms, including **Claude Haiku 4.5** [2], **DeepSeek-V3.2** [5], **GPT-5-mini** [17], **Bandit** [18] (Developed by the OpenStack Security Team), **Semgrep** [21] and **AI-Infra-Guard** [24] (Released by Tencent Zhuque Lab).

**Configuration.** We set the generation temperature to 0.0, disable streaming, and use a 120-second timeout per API request. The evaluation employs dual prompts: a system prompt defining the five MCP vulnerability types, and a user prompt containing the raw Python code of the target server. Security tools (Bandit and AI-Infra-Guard) are run with default settings. To prevent models from exploiting sample order, we shuffle the JSONL dataset with random.shuffle() and a fixed seed, ensuring each sample is evaluated independently. To ensure experimental fairness, we disabled the dynamic penetration testing components of AI-Infra-Guard, and

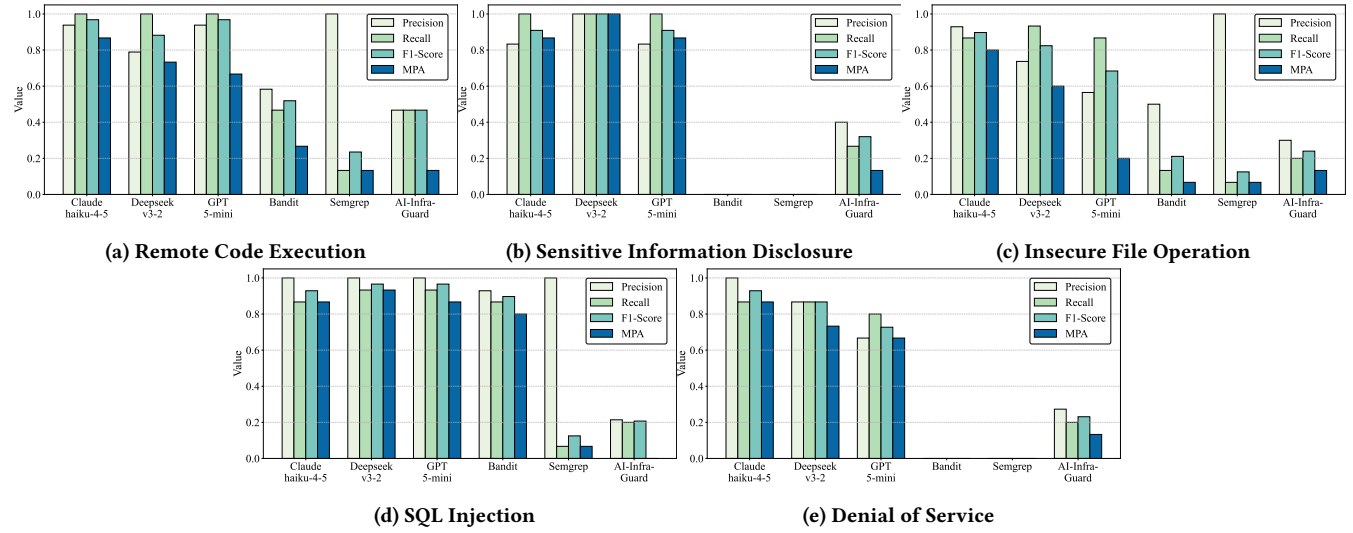


Figure 2: Experimental Results w.r.t MCP Tool Vulnerability Types

retained only its static analysis functionality in our experimental setting.

**Prompt Design.** We use a zero-shot, role-playing prompting strategy for LLM evaluation, separating the prompt into a system and a user prompt to ensure clarity and output stability. The system prompt assigns the role of an “expert security researcher” for Python MCP auditing and embeds the full vulnerability taxonomy, including code-level indicators across five categories. Responses are strictly constrained to a JSON schema with `is_vulnerable`, `vulnerability_type`, and `reasoning` fields. The user prompt provides only the raw Python source code.

### 3.2 Results

Table 2 shows the overall evaluation results. Claude Haiku-4-5 achieves the highest MPA (0.853), reflecting strong ability to distinguish vulnerable MCP servers from their fixed counterparts, while GPT-5-mini’s MPA (0.653) drops significantly below its F1-score (0.842), indicating difficulty in correctly pairing vulnerable and fixed instances. Static tools like Bandit and Semgrep show even larger gaps. Figure 2 illustrates the performance of the evaluated methods across different vulnerability types. Overall, all three LLMs struggle to detect MCP tool vulnerabilities related to insecure file operations, while industrial analysis platforms perform poorly in identifying sensitive information disclosure and denial-of-service vulnerabilities. The MPA is consistently lower than precision, recall, and f1-score across all vulnerability types. On average, all evaluated detectors achieve the lowest MPA on insecure file operation (0.312).

Our experiments reveal the limitations of existing detectors. Furthermore, MPA effectively captures a detector’s semantic understanding and exposes shortcomings in fine-grained vulnerability reasoning that traditional metrics often overlook.

## 4 Related Work

**MCP Benchmarks.** Lin et al. [14] and Wu et al. [27] develop large-scale maintainable and deployable collections of MCP servers. Song et al. [23] introduce MCPGAUGE, a benchmark designed

to systematically evaluate how LLMs utilize MCP tools. Wu et al. [28] propose MCPMark, which assesses multi-step workflows with CRUD operations using expert-curated tasks. Yang et al. [30] present MCPSecBench, integrating a taxonomy of 17 attack types. Additionally, Zhang et al. [31] develop MSB, the first end-to-end evaluation suite measuring LLM agents’ robustness against MCP-specific attacks. Fan et al. [6] propose MCPToolBench++, covering a wide range of single- and multi-step interaction tasks.

**MCP Security.** Song et al. [22] identify novel MCP attack vectors, including tool poisoning, puppet attacks, rug pulls, and exploitation via malicious external resources. Expanding on MCP vulnerabilities, Wang et al. [26] uncover a new Preference Manipulation Attack. Addressing the need for systematic defense mechanisms, Jing et al. [11] develop a fine-grained taxonomy and benchmark to detect and mitigate unsafe behaviors in LLM-MCP interactions. Bühler et al. [3] introduce AgentBound, an access control framework combining declarative, Android-inspired policies with a lightweight enforcement engine. In a similar vein, Kumar et al. [12] propose MCP Guardian, enhancing MCP interactions with authentication, rate-limiting, logging, tracing, and WAF scanning. Xing et al. [29] develop MCP-Guard, a layered defense system for LLM-MCP interactions that integrates static analysis, neural detection, and an LLM-based arbitrator.

## 5 Conclusion

We present a dedicated benchmark for evaluating vulnerability detection in pairing MCP servers with regard to five popular vulnerability types in the MCP servers. We introduce the matched pair accuracy metric, requiring the detectors to correctly classify both the vulnerable instance and its corresponding fixed version simultaneously. Our experimental results reveal previously overlooked limitations in existing techniques, particularly in their ability to distinguish semantically similar vulnerable and non-vulnerable code.

## References

- [1] anonymous. 2026. *Replication Package*. Retrieved February 20, 2026 from <https://anonymous.openscience.r/Benchmark-Data-A1-BED7/>
- [2] anthropic. 2026. *Claude Haiku*. Retrieved April 20, 2026 from <https://www.anthropic.com/news/claude-haiku-4-5>
- [3] Christoph Bühler, Matteo Biagiola, Luca Di Grazia, and Guido Salvaneschi. 2025. Securing AI Agent Execution. *arXiv preprint arXiv:2510.21236* (2025).
- [4] Christian Coleman. 2025. Behavioral Detection Methods for Automated MCP Server Vulnerability Assessment. (2025).
- [5] DeepSeek-AI, Aixin Liu, Aoxue Mei, Bangcai Lin, Bing Xue, Bing-Li Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenhao Xu, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Erhang Li, Fangqi Zhou, Fangyun Lin, Fucong Dai, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Hanwei Xu, Hao Li, Haofen Liang, Haoran Wei, Haowei Zhang, Hao sheng Luo, Haozhe Ji, Honghui Ding, Hongxuan Tang, Huan Cao, Huazuo Gao, Huixian Qu, Hui Zeng, Jialiang Huang, Jiashi Li, Jiaxin Xu, Jiewen Hu, JingChang Chen, Jingting Xiang, Jingyang Yuan, Jing Cheng, Jinhua Zhu, Jun Ran, Junduang Jiang, Junjie Qiu, Junlong Li, Jun-Mei Song, Kai Dong, Kaige Gao, Kang Guan, Kexin Huang, Kexing Zhou, Ke wei Huang, Kuai Yu, Lean Wang, Lecong Zhang, Lei Wang, Liang Zhao, Liangsheng Yin, Lihua Guo, Ling-Li Luo, Linwang Ma, Litong Wang, Liyue Zhang, M. S. Di, M. Y. Xu, Mingchuan Zhang, Minghua Zhou, Min Tang, Mingxu Zhou, P. Huang, Peixin Cong, Peiyi Wang, Qiancheng Wang, Qihao Zhu, Qingyang Li, Qinyu Chen, Qiusi Du, Ruiling Xu, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, Runqiu Yin, Runxin Xu, Ruomeng Shen, Ruoyu Zhang, S. H. Liu, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shaofei Cai, Shaoyuan Chen, Shengding Hu, Shengyu Liu, Shiqiang Hu, Shirong Ma, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, Songyang Zhou, Tao Ni, Tao Yun, Tian Pei, Tian Ye, Tianyuan Yue, Wangding Zeng, Wen Liu, Wenfeng Liang, Wenjie Pang, Wenjing Luo, Wenjun Gao, Wentao Zhang, Xi Gao, Xiangwen Wang, Xiaoling Bi, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaokang Zhang, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xingkai Yu, Xingyou Li, Xinyu Yang, Xinyuan Li, Xu Chen, Xuecheng Su, Xuehai Pan, Xuheng Lin, Xuwei Fu, Y. Q. Wang, Yang Zhang, Yanhong Xu, Yanru Ma, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Qian, Yingpu Yu, Yichao Zhang, Yifan Ding, Yifan Shi, Yi Xiong, Ying He, Ying Zhou, Yinmin Zhong, Yishi Piao, Yisong Wang, Yixiao Chen, Yixuan Tan, Yixuan He, Yiyang Ma, Yiyuan Liu, Yonglun Yang, Yongqiang Guo, Yontong Wu, Yu Wu, Yuan Cheng, Yuan Ou, Yuanfan Xu, Yudian Wang, Yue Gong, Yuhua Wu, Yu-Hui Zou, Yukun Li, Yunfan Xiong, Yu-Wei Luo, Yu mei You, Yuxuan Liu, Yuyang Zhou, Z. F. Wu, Zehui Ren, Zehua Zhao, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhen guo Zhang, Zhewen Hao, Zhibin Gou, Zhicheng Ma, Zhigang Yan, Zhihong Shao, Zhixian Huang, Zhiyu Wu, Zhuoshu Li, Zhuping Zhang, Zian Xu, Zihao Wang, Zihui Gu, Zijia Zhu, Zi-Rui Li, Zipeng Zhang, Ziwei Xie, Ziyi Gao, Zizheng Pan, Zongqing Yao, Bei Feng, Hui Li, J. L. Cai, Jiaqi Ni, Lei Xu, Meng Li, Ning Tian, R. J. Chen, Ruiqi Jin, S. S. Li, Shuang Zhou, Tianyu Sun, X. Q. Li, Xiangyu Jin, Xiaojin Shen, Xiaoshan Chen, Xinnan Song, Xinyi Zhou, Y. X. Zhu, Yanping Huang, Yao Li, Yi Zheng, Yuchen Zhu, Yunxiang Ma, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, Dong-Li Ji, Jian Liang, Jianzhong Guo, Jin Chen, Leyi Xia, Miaojun Wang, Mingming Li, Peng Zhang, Ruyi Chen, Shangmian Sun, Shao-Kang Wu, Sheng-Ying Ye, TWang, W. L. Xiao, Wei An, Xianzu Wang, Xiaowen Sun, Xiaoxiang Wang, Ying Tang, Yukun Zha, Ze-Na Zhang, Zhenghua Ju, Zhen Zhang, and Zihua Qu. 2025. DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models. *ArXiv abs/2512.02556* (2025). <https://api.semanticscholar.org/CorpusID:283448719>
- [6] Shiqing Fan, Kichen Ding, Liang Zhang, and Linjian Mo. 2025. Mcptoolbench++: A large scale ai agent model context protocol mcp tool use benchmark. *arXiv preprint arXiv:2508.07575* (2025).
- [7] google. 2026. *Google Gemini*. Retrieved April 20, 2026 from <https://gemini.google.com>
- [8] Mohammed Mehedi Hasan, Hao Li, Emad Fallahzadeh, Gopi Krishnan Rajbahadur, Bram Adams, and Ahmed E Hassan. 2025. Model context protocol (mcp) at first glance: Studying the security and maintainability of mcp servers. *arXiv preprint arXiv:2506.13538* (2025).
- [9] Xinyi Hou, Shenao Wang, Yifan Zhang, Ziluo Xue, Yanjie Zhao, Cai Fu, and Haoyu Wang. 2026. SMCP: Secure Model Context Protocol. *arXiv preprint arXiv:2602.01129* (2026).
- [10] Xinyi Hou, Yanjie Zhao, Shenao Wang, and Haoyu Wang. 2025. Model context protocol (mcp): Landscape, security threats, and future research directions. *arXiv preprint arXiv:2503.23278* (2025).
- [11] Huihao Jing, Haoran Li, Wenbin Hu, Qi Hu, Xu Heli, Tianshu Chu, Peizhao Hu, and Yangqiu Song. 2025. Mcip: Protecting mcp safety via model contextual integrity protocol. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. 1177–1194.
- [12] Sonu Kumar, Anubhav Girdhar, Ritesh Patil, and Divyansh Tripathi. 2025. Mcp guardian: A security-first layer for safeguarding mcp-based ai system. *arXiv preprint arXiv:2504.12757* (2025).
- [13] latentview. 2026. *Matched-Pair Analysis*. Retrieved February 20, 2026 from <https://www.latentview.com/blog/matched-pair-analysis/>
- [14] Zhiwei Lin, Bonan Ruan, Jiahao Liu, and Weibo Zhao. 2025. A Large-Scale Evolvable Dataset for Model Context Protocol Ecosystem and Security Analysis. *arXiv preprint arXiv:2506.23474* (2025).
- [15] National Institute of Standards and Technology (NIST). 2026. National Vulnerability Database (NVD). <https://nvd.nist.gov/>. Accessed: 2026-01-08.
- [16] nist. 2026. *CVE-2025-5276 Detail*. Retrieved February 20, 2026 from <https://nvd.nist.gov/vuln/detail/cve-2025-5276>
- [17] openai. 2026. *GPT 5 Mini*. Retrieved April 20, 2026 from <https://developers.openai.com/api/docs/models/gpt-5-mini>
- [18] openstack. 2026. *Bandit*. Retrieved April 20, 2026 from <https://wiki.openstack.org/wiki/Security/Projects/Bandit>
- [19] owasp. 2026. *OWASP LLM Top 10*. Retrieved February 20, 2026 from <https://genai.owasp.org/llm-top-10/>
- [20] Brandon Radosevich and John Halloran. 2025. Mcp safety audit: Llms with the model context protocol allow major security exploits. *arXiv preprint arXiv:2504.03767* (2025).
- [21] semgrep. 2025. *semgrep: Meet Your NewAI AppSec Engineer*. Retrieved April 24, 2025 from <https://semgrep.dev/>
- [22] Hao Song, Yiming Shen, Wenxuan Luo, Leixin Guo, Ting Chen, Jiashui Wang, Beibei Li, Xiaosong Zhang, and Jiachi Chen. 2025. Beyond the protocol: Unveiling attack vectors in the model context protocol ecosystem. *arXiv preprint arXiv:2506.02040* (2025).
- [23] Wei Song, Haonan Zhong, Ziqi Ding, Jingling Xue, and Yuekang Li. 2025. Help or Hurdle? Rethinking Model Context Protocol-Augmented Large Language Models. *arXiv preprint arXiv:2508.12566* (2025).
- [24] Tencent. 2026. *AI Infra Guard*. Retrieved April 20, 2026 from <https://github.com/Tencent/AI-Infra-Guard>
- [25] Bin Wang, Zexin Liu, Hao Yu, Ao Yang, Yenan Huang, Jing Guo, Huangsheng Cheng, Hui Li, and Huiyu Wu. 2025. Mcpguard: Automatically detecting vulnerabilities in mcp servers. *arXiv preprint arXiv:2510.23673* (2025).
- [26] Zihan Wang, Rui Zhang, Yu Liu, Wenshu Fan, Wenbo Jiang, Qingchuan Zhao, Hongwei Li, and Guowen Xu. 2025. Mpma: Preference manipulation attack against model context protocol. *arXiv preprint arXiv:2505.11154* (2025).
- [27] Mengying Wu, Pei Chen, Geng Hong, Baichao An, Jinsong Chen, Binwang Wan, Xudong Pan, Jiarun Dai, and Min Yang. 2025. MCPZoo: A Large-Scale Dataset of Runnable Model Context Protocol Servers for AI Agent. *arXiv:2512.15144*
- [28] Zijian Wu, Xiangyan Liu, Xinyuan Zhang, Lingjun Chen, Fanqing Meng, Lingxiao Du, Yiran Zhao, Fanshi Zhang, Yaoqi Ye, Jiawei Wang, et al. 2025. MCPMark: A Benchmark for Stress-Testing Realistic and Comprehensive MCP Use. *arXiv preprint arXiv:2509.24002* (2025).
- [29] Wenpeng Xing, Zhonghao Qi, Yupeng Qin, Yilin Li, Caini Chang, Jiahui Yu, Changting Lin, Zhenzhen Xie, and Meng Han. 2025. Mcp-guard: A defense framework for model context protocol integrity in large language model applications. *arXiv preprint arXiv:2508.10991* (2025).
- [30] Yixuan Yang, Daoyuan Wu, and Yufan Chen. 2025. Mcpsecbench: A systematic security benchmark and playground for testing model context protocols. *arXiv preprint arXiv:2508.13220* (2025).
- [31] Dongsun Zhang, Zekun Li, Xu Luo, Xuannan Liu, Peipei Li, and Wenjun Xu. 2025. MCP Security Bench (MSB): Benchmarking Attacks Against Model Context Protocol in LLM Agents. *arXiv preprint arXiv:2510.15994* (2025).